Hybrid Hierarchical RAG System – Improvement Roadmap

1. Problem Statement

Current hybrid RAG implementations typically combine vector retrieval and structural indexing but lack a clear unified architecture.

This results in inconsistent retrieval, poor ranking, duplicated context, and scalability issues when the document corpus grows.

The goal is to transform the current prototype into a robust hybrid hierarchical retrieval architecture capable of handling

large document collections efficiently and accurately.

2. Core Architectural Principle

All retrieval layers must operate on a unified document node structure rather than independent chunk formats.

Every piece of information should be represented as a node in a document graph/tree structure.

Example Node Schema:

Node {

id

content

type (document, section, paragraph, summary)

parent

children

embedding (optional)

keywords

metadata

}

3. Multi-Layer Retrieval Architecture

The system should support multiple retrieval mechanisms operating on the same node index.

Vector Retrieval:

- Used for semantic similarity search

- Embeddings stored for selected nodes

Keyword Retrieval (BM25):

- Exact keyword matching

- Important for technical queries and precise search

Tree Retrieval:

- Structural navigation through document hierarchy

- Expands relevant sections and neighboring nodes

ARCHITECTURE DIAGRAM

This is how your project should look in documentation.

```
                USER QUERY
                    |
            Query Understanding
                    |
              Query Classification
                    |
        ┌───────────┼───────────┐
        |           |           |
Vector Search   Tree Search   Keyword Search
        |           |           |
        └───────────┼───────────┘
                    |
                    |
             Candidate Pool
                    |
             Hybrid Reranker
                    |
            Context Optimization
                    |
                   LLM
                    |
                 RESPONSE
```

4. Query Understanding Layer

Before retrieval, the system should classify the query type.

Example categories:

- Definition queries

- Comparison queries

- Reasoning queries

- Summarization queries

- Navigation queries

A lightweight LLM or rule-based classifier can determine the query intent and guide retrieval strategy selection.

5. Multi-Stage Retrieval Pipeline

Recommended pipeline:

User Query

→ Query Understanding

→ Vector Retrieval

→ Tree Retrieval

→ Keyword Retrieval

→ Candidate Node Pool

→ Hybrid Reranking

→ Context Optimization

→ LLM Response Generation

6. Hybrid Reranking Strategy

Retrieved nodes must be ranked using a combined scoring function.

Example scoring model:

Score =

0.5 × Semantic Similarity

+ 0.3 × Structural Proximity

+ 0.2 × Keyword Relevance

This ensures fair comparison across retrieval sources.

7. Context Optimization Layer

Instead of sending raw retrieved chunks directly to the LLM, implement a context optimization step.

Steps:

1. Remove duplicate nodes
2. Cluster related nodes
3. Merge overlapping context
4. Summarize clusters if necessary
5. Construct final coherent context

This reduces noise and improves answer accuracy.

8. Tree Expansion Retrieval

After retrieving a relevant node, expand its neighbors in the document tree.

Example:

Section: Transformers

→ Subsection: Architecture

→ Subsection: Training

→ Subsection: Advantages

This preserves document coherence and contextual completeness.

9. Retrieval Memory (Adaptive Retrieval)

Introduce a learning mechanism that improves retrieval over time.

Workflow:

Query

→ Retrieved nodes

→ User feedback / evaluation

→ Update ranking weights

The system gradually learns which retrieval strategies work best for different query types.

10. Context Conflict Detection

Detect contradictions among retrieved nodes before sending context to the LLM.

Example:

Node A: Python created in 1990

Node B: Python created in 1991

Use LLM-based contradiction detection to flag inconsistent information and remove unreliable context.

11. Scalable Embedding Strategy

Avoid embedding every small chunk in the corpus.

Recommended approach:

Embed only important nodes such as:

- document summaries

- section summaries

- key paragraphs

Benefits:

- smaller vector index

- faster retrieval

- reduced memory consumption

12. Unified Node-Based Indexing

All retrieval systems must reference the same node IDs.

Vector Index:

node_id → embedding

Keyword Index:

node_id → token list

Tree Index:

node_id → parent/child relations

This prevents duplication and fragmentation across retrieval layers.

13. Evaluation and Benchmarking

To validate improvements, evaluate the system on established QA benchmarks.

Recommended datasets:

- HotpotQA

- NaturalQuestions

- FiQA

Metrics:

- retrieval accuracy

- answer accuracy

- latency

- context size

14. Future Extensions

Potential advanced improvements:

- Graph-based knowledge extraction (GraphRAG)

- Late interaction retrieval (ColBERT-style search)

- Agentic multi-step retrieval loops

- Cross-document reasoning chains